

Phần 8 - Case Studies

Mục lục

8.1 Tổng quan Phần 8: Case Studies	5
Nếu bạn đến từ Data Science	5
Vì sao case study?	5
3 Case Studies	5
Case 1: UAMS, University Academic Management System	5
Case 2: Smart City Traffic Incident Detection	5
Case 3: Production ML Feature Store	5
Bài tập	6
Cách áp dụng framework cho mỗi case	6
Bước 1: Hiểu domain + functional requirements (Phần 1)	6
Bước 2: Identify Quality Attributes (Phần 4)	6
Bước 3: Cân nhắc Style, Monolithic vs Distributed (Bài 5.2)	6
Bước 4: Choose specific Architecture Style (Phần 5-6)	6
Bước 5: Component decomposition (Bài 2.3 SRP + Bài 3.4 + Bài 4.4)	6
Bước 6: Document (Phần 7)	6
Sản phẩm cuối case	6
Tone của cases	7
Bài trong phần	7
8.2 UAMS, University Academic Management System	8
1. Domain và Functional Requirements	8
Tổ chức bối cảnh	8
Core features	8
Users	8
Usage patterns	8
2. Identify Quality Attributes	8
Top 7 QA	8
Implicit characteristics	9
Operationalize	9
3. Monolithic vs Distributed	9
Considerations	9
Decision	9
4. Architecture Style Selection	10
Style choice: Service-based + selective async	10
Bounded contexts □ services	10
5. Component Decomposition	11
Inside Enrollment Service (example)	11

6. Documenting Key Decisions (ADRs)	11
ADR-001: Use Service-based over Microservices	11
ADR-002: Shared PostgreSQL với schema-per-service	12
ADR-003: Sync HTTP for inter-service	12
ADR-004: Audit log via DB trigger	12
ADR-005: Auto-scale Enrollment Service for registration week	12
7. Critical Flows	13
Flow 1: Student registers for course	13
Flow 2: Instructor submits grade	13
8. QA Scorecard	13
9. Trade-off Reflection	14
10. What we'd do differently	14
Tóm tắt	14
8.3 Smart City Traffic Incident Detection	15
1. Domain và Functional Requirements	15
Background	15
Data sources (heterogeneous)	15
Functional requirements	15
Scale calculation	15
Góc nhìn Data Scientist	15
2. Identify Quality Attributes	16
Top 7 QA	16
Operationalize	16
3. Monolithic vs Distributed	16
Considerations	16
Decision	17
4. Architecture Style	17
Style mix	18
5. Component Decomposition	18
Ingestion services (1 per source type)	18
Normalizer services	18
Detection services (microkernel plug-ins)	18
Storage	18
6. Key ADRs	19
ADR-001: Kafka as event backbone	19
ADR-002: Microkernel for detection services	19
ADR-003: TimescaleDB + BigQuery split	19
ADR-004: At-least-once delivery with idempotent consumers	20
ADR-005: Geo-sharded Kafka topics	20
7. Critical Flow	20
Detection of accident from camera frame	20
8. QA Scorecard	20
9. Trade-off Reflection	21
10. So sánh UAMS vs Smart City	21
Tóm tắt	21
8.4 Production ML Feature Store	22
1. Vì sao case này quan trọng?	22

2. Domain và bối cảnh	22
3. Functional Requirements	23
Feature management	23
Training support	23
Serving support	23
Monitoring and governance	23
4. Constraints và scale	23
5. Identify Quality Attributes	24
Top QA	24
Operationalize	24
6. Architecture style decision	25
Options considered	25
Decision	25
7. High-level Architecture	25
8. Component decomposition	25
9. Key runtime flows	27
Flow A: Register a feature	27
Flow B: Build training dataset	28
Flow C: Online prediction	28
Flow D: Streaming feature update	29
10. Data contracts	29
11. Training-serving skew	29
Skew scenario	29
Architecture fix	30
12. Privacy and governance	30
Policy examples	30
Design implication	31
13. Observability design	31
14. Failure modes and mitigations	31
15. ADR examples	31
ADR 1: Use dual offline and online feature stores	31
ADR 2: Support freshness tiers instead of realtime for all features	32
ADR 3: Choose service-based core, not full microservices	32
16. Cost model rough estimate	32
17. Evolution roadmap	32
Phase 1: Batch-first feature platform	32
Phase 2: Online serving	33
Phase 3: Streaming and governance	33
18. What a Data Scientist should learn from this case	33
19. Self-check	33
20. Tóm tắt	33
8.5 Bài tập tổng hợp	35
Cách làm	35
Bài 1: Online Food Delivery Platform	35
Context	35
Functional requirements	35
Constraints	35
Hints	35

Things to consider	36
Output expected	36
Bài 2: B2B SaaS for Property Management	36
Context	36
Functional requirements	36
Constraints	36
Hints	36
Things to consider	37
Output expected	37
Bài 3: Real-time Stock Trading Platform	37
Context	37
Functional requirements	37
Constraints	37
Hints	37
Things to consider	37
Output expected	38
Bài 4: IoT Manufacturing Monitoring	38
Context	38
Functional requirements	38
Constraints	38
Hints	38
Things to consider	38
Bài 5: Multi-region E-commerce Platform	38
Context	38
Functional requirements	39
Constraints	39
Hints	39
Things to consider	39
Bài 6: Churn Prediction Platform	39
Context	39
Functional requirements	39
Constraints	39
Questions	40
Bài 7: Real-time Fraud Detection	40
Context	40
Functional requirements	40
Constraints	40
Questions	40
Bài 8: Feature Store cho ML Platform	40
Context	40
Functional requirements	40
Constraints	41
Questions	41
Reflection questions	41
Discussion với mentor (nếu có)	41
Tóm tắt	41
Tổng kết Phần 8 và toàn khoá	41

8.1 Tổng quan Phần 8: Case Studies

Tóm tắt một dòng: 7 phần trước cung cấp building block. Phần này áp tất cả vào 3 case thực để bạn thấy cách lập luận “given context X, why this architecture not that” - skill thực sự của architect.

Nếu bạn đến từ Data Science

Trong ba case study, hãy đọc kỹ Smart City Traffic và Production ML Feature Store. Smart City cho bạn thấy realtime data/ML system với ingest, normalize, streaming, ML detector, alerting, time-series storage và data warehouse. Production ML Feature Store đi sát hơn vào công việc DS/MLOps: feature definition, offline store, online store, model registry, serving API, drift monitoring và auditability.

UAMS vẫn đáng đọc vì nó dạy phần business workflow, audit, data integrity và service-based thinking. Nhưng nếu mục tiêu của bạn là đưa model vào production, hãy ưu tiên 8.3 và 8.4.

Vì sao case study?

Lý thuyết kiến trúc dễ học, áp dụng khó. Phần 1-7 cho bạn vocabulary và principles. Nhưng câu hỏi thực sự khi làm architect là:

- “Hệ này nên monolith hay microservices?”
- “Database nào? SQL hay NoSQL?”
- “Communication nào? Sync HTTP hay async event?”
- “Architecture style nào best fit?”

Câu trả lời tùy context. Case study đi qua **lập luận end-to-end** cho ba context khác nhau, show cách architect thinks.

3 Case Studies

Case 1: UAMS, University Academic Management System

Context:

- Trường đại học multi-program (undergraduate, postgraduate).
- Centralized system manage student, course, grade.
- Regulatory compliance (data privacy, audit trail).
- Scale: 50k student peak (registration week).

Focus: structured business processes, ACID requirements, internal users.

Case 2: Smart City Traffic Incident Detection

Context:

- City government deploy system phát hiện sự cố giao thông real-time.
- Data sources: camera, sensor, GPS, citizen mobile reports.
- Notify operator, emergency services, navigation systems.
- Scale: thousands sensors, millions data points/hour, sub-second response.

Focus: high-throughput streaming, real-time, heterogeneous data.

Case 3: Production ML Feature Store

Context:

- Công ty SaaS có nhiều team DS/ML cùng build model production.
- Duplicate feature logic, training-serving skew, thiếu lineage và monitoring.
- Cần feature registry, offline store, online store, model registry, serving API.
- Scale: 40 models production, 200 features tăng lên 1000, 15k RPS peak.

Focus: production ML, feature freshness, online inference, auditability, privacy, service-based + pipeline + event-driven mix.

Bài tập

Cuối phần có bộ exercises để bạn tự apply.

Cách áp dụng framework cho mỗi case

Mỗi case sẽ đi qua 6 bước (synthesizing các phần trước):

Bước 1: Hiểu domain + functional requirements (Phần 1)

Đào ra hệ làm gì, ai dùng, business context.

Bước 2: Identify Quality Attributes (Phần 4)

Top 5-7 QA priority. Operationalize.

Bước 3: Cân nhắc Style, Monolithic vs Distributed (Bài 5.2)

Default monolithic; chỉ distributed nếu justify.

Bước 4: Choose specific Architecture Style (Phần 5-6)

Match QA priorities với style trade-offs.

Bước 5: Component decomposition (Bài 2.3 SRP + Bài 3.4 + Bài 4.4)

Bounded context, module boundary.

Bước 6: Document (Phần 7)

C4 Context + Container, key sequence, ADR for important decisions.

Sau mỗi case sẽ có “What we’d do differently”, phần phản biện.

Sản phẩm cuối case

Mỗi case xuất ra:

1. **System Context diagram** (C4 Level 1).
2. **Container diagram** (C4 Level 2).
3. **2-3 sequence diagram** cho critical flows.
4. **3-5 ADR** cho key decisions.
5. **QA scorecard**: bảng QA vs measurement.
6. **Trade-off reflection**: “đã hy sinh gì để đạt gì”.

Khi bạn làm xong, output này là *template* cho architecture doc của project bạn.

Tone của cases

Cases viết theo phong cách “consulting walk-through”, như consultant trình bày cho client. Bạn đọc như đang sit-in một consulting engagement.

Mỗi case dài ~3500-5000 chữ, đọc 1.5-2h. Đáng dành thời gian.

Bài trong phần

- **8.2 UAMS**: Academic Management System.
- **8.3 Smart City Traffic**: Real-time incident detection.
- **8.4 Production ML Feature Store**: feature store, model serving, monitoring, governance.
- **8.5 Exercise Set**: 8 bài tập tự practice với hint.

Vào [Bài 8.2: UAMS](#) để bắt đầu, hoặc nếu bạn đến từ Data Science, có thể đi thẳng tới [Bài 8.4: Production ML Feature Store](#).

8.2 UAMS, University Academic Management System

Tóm tắt một dòng: Case study lập luận end-to-end cho hệ academic management. Conclusion - Service-based architecture với 6 services, shared PostgreSQL với schema-per-service, sync HTTP cho internal call, sync API + sync admin UI. Tổng cost dự kiến \$1500/tháng cho 50k students.

1. Domain và Functional Requirements

Tổ chức bối cảnh

Một trường đại học chạy đa chương trình (undergraduate, postgraduate). Quyết định xây dựng centralized Academic Management System để chuẩn hoá operations và đảm bảo compliance regulatory.

Core features

- **Student enrollment** + profile management.
- **Course + curriculum management.**
- **Semester registration** (sinh viên đăng ký môn).
- **Grade submission** + GPA calculation.
- **Role-based access control** (student, instructor, registrar, admin).

Users

- **Students** (peak 50k, average 30k active).
- **Instructors** (~2000).
- **Registrars** (~50 staff).
- **Admins** (~10).

Usage patterns

- **Regular:** spread throughout semester, modest load (100-500 req/min).
- **Registration week:** 10x normal. 50k student đăng ký môn trong 2-3 ngày.
- **Grade submission week:** bursty từ instructor side.

2. Identify Quality Attributes

Workshop với stakeholder (Registrar Director, IT Director, Dean of Studies):

Top 7 QA

Priority	QA	Target
Must	Data integrity	100% (registration không có double-booking, grade chính xác)
Must	Security	Compliance regulatory (FERPA equivalent). Audit log mọi grade change

Priority	QA	Target
Must	Availability	99.5% (4 hours downtime/month acceptable, except registration week)
Must	Auditability	Full trace của ai sửa grade/enrollment khi nào
Should	Scalability	Handle 10x load in registration week
Should	Maintainability	New feature ship trong 1-2 sprint
Should	Cost	< \$3000/month infrastructure cho 50k students

Implicit characteristics

- Privacy (data sinh viên).
- Performance acceptable (page load < 2s).
- Onboard new staff < 1 week (admin UI usable).

Operationalize

QA	Metric	Tool
Data integrity	0 inconsistency in audit (quarterly)	DB constraint, automated reconciliation
Security	OWASP Top 10 quarterly pen-test	External pen-test
Availability	Uptime % from monitoring	Datadog/UptimeRobot
Auditability	100% grade changes have audit log	DB trigger + audit table
Scalability	Handle 50k concurrent registration	Load test pre-registration week
Maintainability	Mean PR-to-prod time < 1 week	Git analytics
Cost	Monthly cloud bill	Cloud billing dashboard

3. Monolithic vs Distributed

Considerations

- **Team size:** ~15 engineers (internal IT). Not huge.
- **Domain:** Medium complexity (~6 bounded contexts).
- **Scale:** 50k users peak. Significant but not enormous.
- **Regulatory:** separate audit log easier in microservices, but monolithic with proper modules also OK.

Decision

Service-based (4-12 coarse services, shared DB per domain). Reasons:

- Team ≈ 15 $\square$ not enough for full microservices (need 50+).
- 6 bounded contexts $\square$ service-based natural fit.
- 10x scaling spike $\square$ service-based scales OK (better than monolith, less complex than microservices).
- Operational team không có K8s expertise $\square$ microservices nặng.

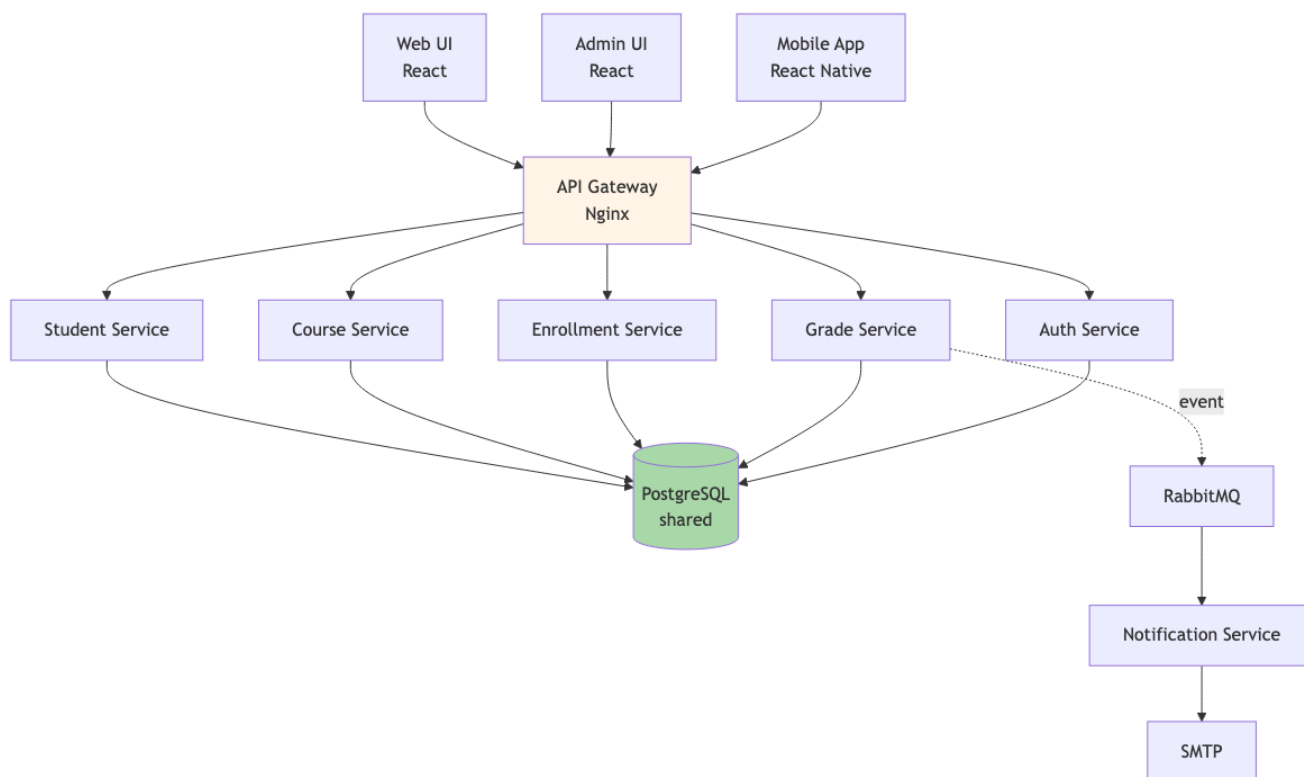
Justify cost-benefit:

- Service-based ops cost $\sim$ \$1500/month vs monolith \$500/month vs microservices \$5000/month.
- Microservices không justify ROI cho team 15 + scale này.

4. Architecture Style Selection

Style choice: Service-based + selective async

- **Sync HTTP (REST)** for internal service calls.
- **Async event** (RabbitMQ) for cross-context notifications (vd: grade submitted $\square$ notify student email).
- **Shared PostgreSQL** with schema-per-service.



Hình 1: Sơ đồ 49

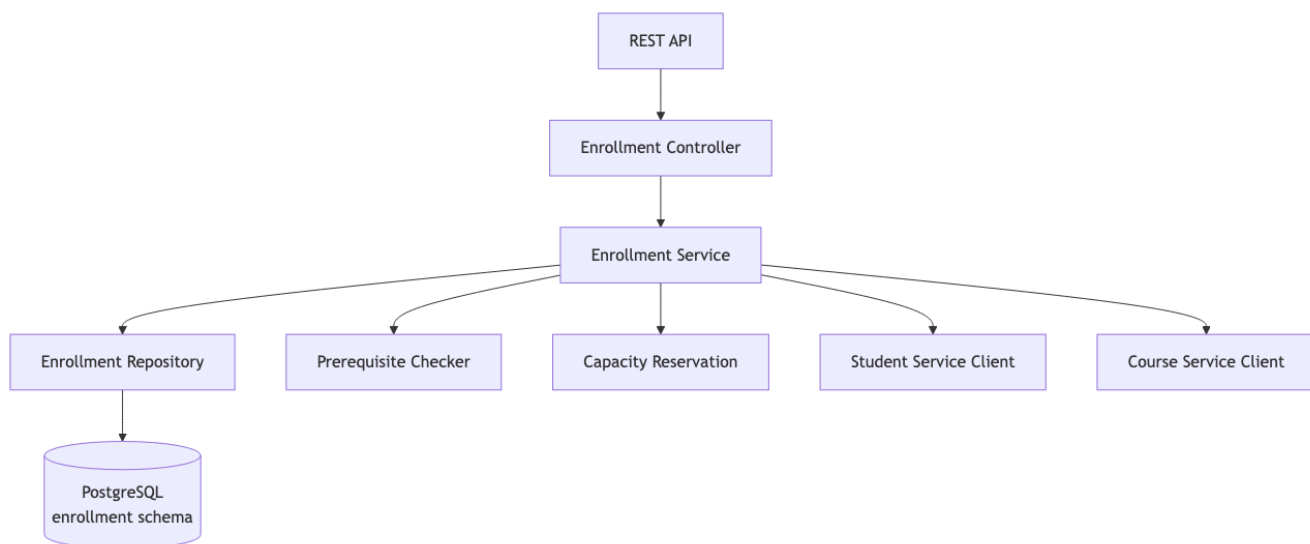
Bounded contexts $\square$ services

Service	Bounded Context	Owns Entities
Student Service	Student domain	Student profile, enrollment history
Course Service	Catalog	Course, Curriculum, Section
Enrollment Service	Registration	Registration, Schedule
Grade Service	Assessment	Grade, GPA calculation
Auth Service	Identity	User, Role, Permission
Notification Service	Communication	Email/SMS queue

6 services, total 15-engineer team (2-3 engineers per service).

5. Component Decomposition

Inside Enrollment Service (example)



Hình 2: Sơ đồ 50

Components inside service follow layered (Bài 5.3).

6. Documenting Key Decisions (ADRs)

ADR-001: Use Service-based over Microservices

Context

Team 15 engineers, 6 bounded contexts, 50k user peak, ops team không có K8s.

Decision

Service-based với 6 coarse services + shared PostgreSQL.

Alternatives Considered

- Monolith: pain với deploy bottleneck dự kiến trong 2 năm.
- Microservices: cost ops 3-5x, team không ready.

Consequences

- Lợi: team velocity, deploy independent, manageable ops.
- Hại: chưa có full team independence như microservices.

ADR-002: Shared PostgreSQL với schema-per-service**## Decision**

Single PostgreSQL instance, schema-per-service (student_svc, course_svc, ...).

Rationale

- Cost: 1 DB instance vs 6 instances.
- Cross-service query: support khi cần (vd: report tổng hợp).
- Compliance audit: single DB easier để audit.
- Trade-off: phải discipline schema migration coordination.

ADR-003: Sync HTTP for inter-service**## Decision**

Sync HTTP (REST) for service-to-service calls, async RabbitMQ for notifications.

Rationale

- Sync: simple, debug dễ, fit cho most user-facing flows.
- Async: notification không cần real-time, decoupling email service.

Consequences

- Cascade failure risk if service down: mitigate by circuit breaker (Resilience4j).
- Latency: 3-5 hops max for any user request, sub-1s achievable.

ADR-004: Audit log via DB trigger**## Decision**

PostgreSQL trigger on grade/enrollment tables → write to audit_log table.

Rationale

- Compliance: full trace required.
- DB-level capture: cannot bypass via app bug.
- Trade-off: DB trigger complexity, performance overhead ~5%.

Alternatives Considered

- Application-level logging: rejected (can be bypassed).
- CDC (Change Data Capture): rejected (extra infrastructure complexity).

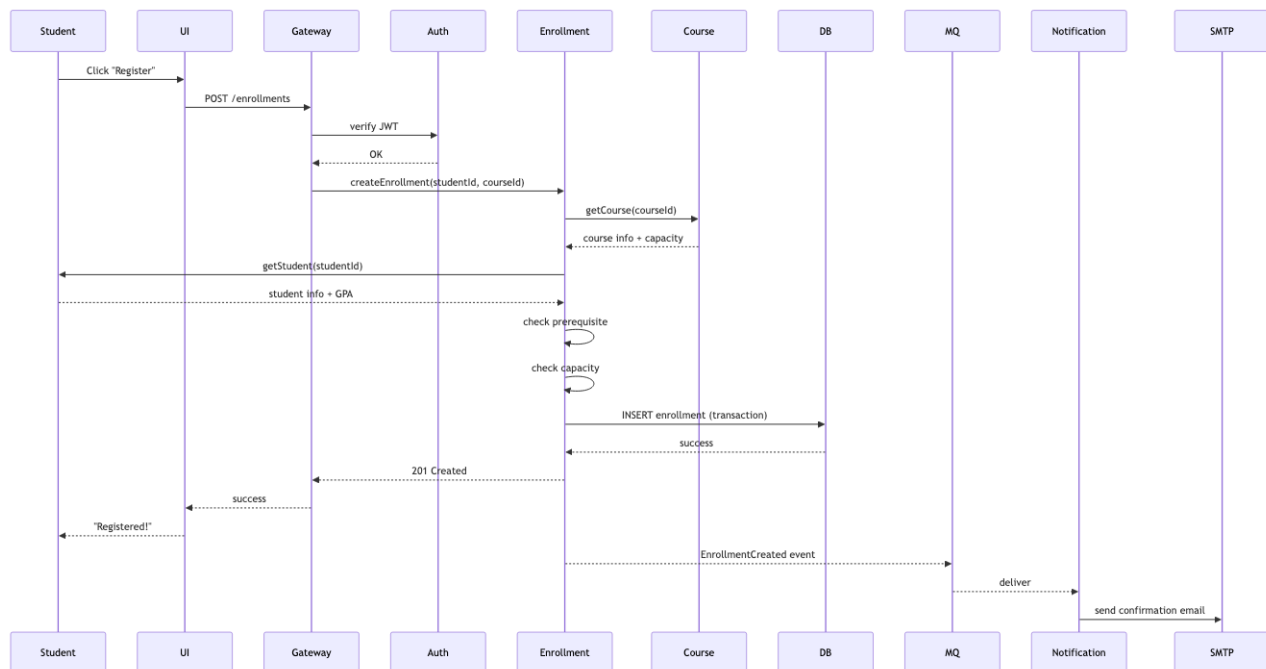
ADR-005: Auto-scale Enrollment Service for registration week

Decision

Auto-scale Enrollment Service (3 → 30 instances) during registration week.
Other services stay 2-3 instances.

Rationale

- 10x load only on Enrollment during registration.
- Cost optimization: scale only what's needed.

7. Critical Flows**Flow 1: Student registers for course**

Hình 3: Sơ đồ 51

Flow 2: Instructor submits grade

(similar pattern, brief)

8. QA Scorecard

QA	Target	How achieved
Data integrity	100%	DB constraints, transaction, automated reconciliation job
Security	OWASP compliant	Auth service, RBAC, prepared statements (SQL injection), HTTPS, secret in Vault
Availability	99.5%	Multi-AZ, RDS HA, service replicas, circuit breakers
Auditability	100% grade changes	DB trigger + audit_log table

QA	Target	How achieved
Scalability	Handle 50k peak	Auto-scale Enrollment, read replicas during peak
Maintainability	PR-to-prod < 1 week	CI/CD per service, contract tests
Cost	< \$3000/month	\$1500 baseline + \$1000 during peak weeks = \$1500-2500/month average

All targets met.

9. Trade-off Reflection

Hy sinh:

- **Full microservices independence:** 6 services share DB, không thể database-per-service migrate hoàn toàn isolated.
- **Polyglot tech stack:** all services Java/Spring (team uniform). Không thể chọn Python cho ML service nếu cần sau này (would require refactor).
- **Sub-100ms latency:** sync calls accumulate. p99 200-500ms typical, acceptable cho academic use.

Đạt:

- **Pragmatic ops:** 1 ops engineer can handle 6 services + 1 DB.
- **Team velocity:** each team owns 1-2 services, deploy independently.
- **Compliance:** audit + RBAC built into architecture.
- **Cost-effective:** \$1500-2500/month for 50k students.

10. What we'd do differently

Honest reflection:

1. **Add event sourcing for Grade:** Audit log good but reconstructing historical grade state hard. Event sourcing for grade entity would simplify.
2. **API versioning từ ngày 1:** Currently API not versioned. When mobile app released, breaking changes painful.
3. **Async cho slow operations:** Bulk grade upload by instructor □ sync HTTP timeout. Should be async with progress tracking from start.
4. **Multi-region for DR:** Single region acceptable for academic, but disaster recovery plan thiếu. Should have replica in second region.

Tóm tắt

UAMS case demonstrates:

- **Service-based** chosen over monolith (team velocity) vs microservices (ops cost).
- **Shared DB schema-per-service** balance isolation + cost.
- **Sync HTTP** for user flows, **async event** for notifications.
- **Audit via DB trigger** for compliance.
- **Auto-scale** specific service for predictable load spike.

Pragmatic, not perfect. Apt for context.

Bài tiếp: [Smart City Traffic Detection](#), context completely different.

8.3 Smart City Traffic Incident Detection

Tóm tắt một dòng: Case study lập luận cho hệ phát hiện sự cố giao thông real-time. Conclusion - Event-Driven Architecture với Kafka pipeline, microservices style cho detection logic, time-series DB cho historical analytics. Khác hoàn toàn UAMS vì context khác.

1. Domain và Functional Requirements

Background

City government deploy Smart Traffic Incident Detection System để cải thiện an toàn đường + giảm congestion.

Data sources (heterogeneous)

- **Roadside cameras** (~5000 cameras, 1 frame/giây mỗi camera).
- **Traffic sensors** (~10,000 sensors đo speed + volume, update mỗi 5 giây).
- **GPS data** từ public buses + taxis (~3000 vehicles, ping mỗi 10 giây).
- **Citizen reports** (mobile app, sporadic, ~100 reports/giờ).

Functional requirements

- Ingest data heterogeneous, high-volume.
- Clean + normalize.
- Detect incidents real-time: accidents, stalled vehicles, congestion (rule-based + AI/ML).
- Generate alerts với severity levels.
- Store data cho historical analysis (~7 years).
- Notify traffic operators, emergency services, navigation systems.

Scale calculation

- Cameras: $5000 \times 1 \text{ frame/s} = 5000 \text{ events/s}$.
- Sensors: $10000 \times 1 \text{ update} / 5\text{s} = 2000 \text{ events/s}$.
- GPS: $3000 \times 1 \text{ ping} / 10\text{s} = 300 \text{ events/s}$.
- Citizen: $\sim 0.03 \text{ events/s}$.

Total: **~7300 events/s** sustained, peak 15-20k events/s.

Per day: ~700 million events. Per year: ~250 billion. Storage massive.

Góc nhìn Data Scientist

Nếu bạn đến từ Data Science, case này nên được đọc như một production ML system hơn là một bài backend thông thường. Model phát hiện tai nạn chỉ là một component ở giữa pipeline. Trước model có ingestion, validation, normalization, feature construction. Sau model có alerting, audit log, dashboard, storage nóng/lạnh và monitoring.

Điểm quan trọng là model không tự quyết định kiến trúc. Kiến trúc bị chi phối bởi các Quality Attributes:

- **Freshness:** dữ liệu camera/sensor/GPS phải đến detector đủ nhanh.
- **Latency:** incident cần được detect trong vài chục giây.
- **Throughput:** hệ phải xử lý hàng nghìn event mỗi giây.

- **Reliability**: không được mất event quan trọng.
- **Auditability**: khi có alert sai, phải truy lại input, model version, feature version.
- **Evolvability**: thêm detector mới mà không dừng toàn hệ thống.

Vì vậy lựa chọn Event-Driven + Pipeline + Microkernel không phải vì “hiện đại”, mà vì nó match với QA của bài toán. Đây là cách bạn nên đọc mọi architecture decision trong case.

2. Identify Quality Attributes

Top 7 QA

Priority	QA	Target
Must	Scalability	Handle 20k events/s sustained, scale to 50k events/s
Must	Low latency (detection)	Incident detected within 30s of event
Must	Low latency (alert)	Alert delivered within 10s of detection
Must	Availability	99.9% (detection must not miss critical incidents)
Must	Extensibility	Add new detection algorithm without redeploy core
Should	Reliability	No data loss in pipeline
Should	Cost	< \$20k/month for current scale

Operationalize

QA	Metric	Tool
Scalability	Events/s throughput, lag	Kafka metrics, Datadog
Detection latency	Time from event to detection	Distributed tracing
Alert latency	Time from detection to notification	Logs
Availability	Uptime %	Multi-region health check
Extensibility	Time to deploy new algorithm	Git analytics
Reliability	Message loss rate	Kafka offset monitoring
Cost	Monthly cloud bill	Cloud billing

3. Monolithic vs Distributed

Considerations

- **Scale**: 7-20k events/s, monolithic single instance không feasible.
- **Heterogeneous sources**: different ingestion logic, parallel.
- **Real-time detection**: cần stream processing.
- **Extensibility**: add detection algo dễ □ plug-in style.

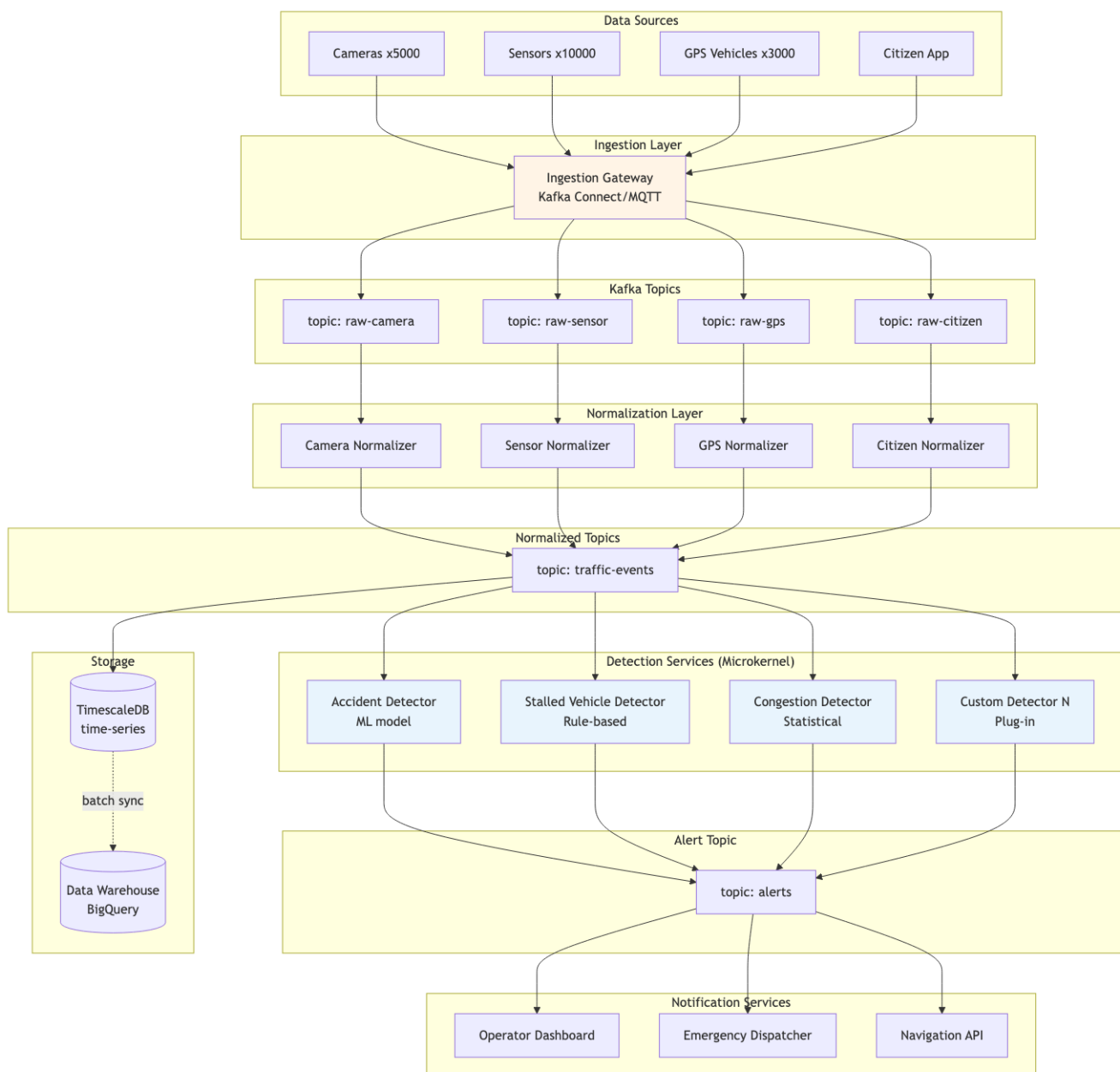
Decision

Distributed, Event-Driven Architecture (Bài 6.4) overlay với **Microservices** (Bài 6.3) cho detection services.

Justify:

- Scale + throughput requirements □ distributed required.
- Async detection chain □ event-driven natural.
- Extensibility for new algorithm □ microservices + Microkernel pattern.

4. Architecture Style



Hình 4: Sơ đồ 52

Style mix

- **Event-Driven (Bài 6.4)**: backbone với Kafka.
- **Microservices (Bài 6.3)**: fine-grained detection services.
- **Microkernel (Bài 5.5)**: detection services là plug-ins to a “detection core”, easy add new algorithm.
- **Pipeline (Bài 5.4)**: ingest $\square$ normalize $\square$ detect là pipeline.

Multiple styles compose. Architect chọn fit-for-purpose per layer.

5. Component Decomposition

Ingestion services (1 per source type)

- Camera Ingestor: MQTT broker $\square$ Kafka.
- Sensor Ingestor: HTTP POST endpoint $\square$ Kafka.
- GPS Ingestor: TCP socket (NMEA protocol) $\square$ Kafka.
- Citizen Ingestor: HTTPS REST API $\square$ Kafka.

Normalizer services

Mỗi normalizer:

- Consume raw Kafka topic.
- Validate, clean, normalize to common schema.
- Publish to traffic-events topic.

Common schema:

```
{
  "event_id": "uuid",
  "timestamp": "ISO 8601",
  "source_type": "camera|sensor|gps|citizen",
  "source_id": "string",
  "location": {"lat": ..., "lon": ..., "road_id": "..."},
  "data": {...source-specific...},
  "metadata": {...}
}
```

Detection services (microkernel plug-ins)

Mỗi detector:

- Subscribes traffic-events topic (filter by location/source).
- Apply detection logic (rule, statistical, ML).
- Publish IncidentDetected event to alerts topic.

New detector = new microservice subscribing topic. Core không thay đổi.

Storage

- **TimescaleDB**: hot data, last 30 days, real-time query.
- **BigQuery**: cold data, 7 years, analytics.

Pipeline batch sync TimescaleDB $\square$ BigQuery daily.

6. Key ADRs

ADR-001: Kafka as event backbone

Decision

Apache Kafka as event broker for entire pipeline.

Rationale

- Throughput: handles 20k+ events/s easily.
- Replay: critical for ML model retraining and debugging.
- Schema registry support.
- Industry standard for streaming.

Alternatives

- RabbitMQ: not designed for stream replay, lower throughput.
- AWS Kinesis: vendor lock, similar capability.
- Pulsar: newer, less ecosystem.

Consequences

- Ops complexity (Kafka cluster management), invest in tooling.
- Need schema registry (Confluent or Apicurio).

ADR-002: Microkernel for detection services

Decision

Detection logic implemented as plug-in microservices that subscribe to traffic-events topic.

Rationale

- New detection algorithms common (city evolves rules).
- Each algorithm runs independently, fault isolation.
- Can be deployed by different teams.

Consequences

- Plug-in API (Kafka schema) must be stable.
- Discovery: each algorithm registers its capabilities.

ADR-003: TimescaleDB + BigQuery split

Decision

TimescaleDB (PostgreSQL extension) for hot 30 days. BigQuery for 7-year history.

Rationale

- Real-time dashboards query last 30 days → need low-latency time-series DB.
- 7-year analytics → batch jobs OK → BigQuery cost-effective for cold data.
- Cost: TimescaleDB ~\$1500/month for 30 days. BigQuery storage cheap, query pay-per-use.

Consequences

- Two systems to manage.

- Sync job complexity.
- Cross-DB query not possible (acceptable).

ADR-004: At-least-once delivery with idempotent consumers

Decision

Kafka at-least-once delivery. Detectors implement idempotency by event_id.

Rationale

- Exactly-once expensive and complex.
- Most detection logic naturally idempotent (same event, same detection).
- Alert deduplication at notification layer.

Consequences

- All detectors must implement idempotency. Code review enforce.

ADR-005: Geo-sharded Kafka topics

Decision

Kafka topics partitioned by geographic region (vd: district 1-12).

Rationale

- Locality: detection algorithms can subscribe to specific districts.
- Scale: parallelize processing by partition.
- Future: regional clusters if needed.

Consequences

- Routing logic in ingester (which partition).
- Re-shard rare but complex.

7. Critical Flow

Detection of accident from camera frame

End-to-end: camera frame to alert ~15-20s. Within target.

8. QA Scorecard

QA	Target	How achieved
Scalability	20k events/s	Kafka partition, horizontal scale detector
Detection latency	30s	Streaming pipeline (vs batch)
Alert latency	10s	Kafka consumer group with low-latency config
Availability	99.9%	Multi-AZ Kafka, replica consumers, multi-region failover
Extensibility	New algorithm in days	Microkernel pattern, well-defined event schema

QA	Target	How achieved
Reliability	No data loss	At-least-once + idempotent consumers
Cost	< \$20k/month	Auto-scale, BigQuery for cold data

9. Trade-off Reflection

Hy sinh:

- **Strong consistency:** events arrive out of order across partition (per-partition ordered). Detection algorithms handle.
- **Operational complexity:** Kafka cluster + many microservices + 2 storage systems = need experienced SRE team.
- **Cost:** \$15-20k/month higher than UAMS but justify by scale.

Đạt:

- **Scalable to 50k+ events/s** with horizontal scaling.
- **Sub-30s detection** real-time.
- **Plug-in extensibility** for new algorithms.
- **7-year audit** via BigQuery.

10. So sánh UAMS vs Smart City

Aspect	UAMS	Smart City
Style	Service-based	Event-Driven + Microservices + Microkernel
DB	Shared PostgreSQL	TimescaleDB + BigQuery
Communication	Sync HTTP + async event	Async event chỉ
Scale	50k peak users	20k events/s sustained
Latency target	< 2s page load	< 30s detection, < 10s alert
Cost	\$1500-2500/month	\$15-20k/month
Team needed	15 engineers	25-30 engineers + 5 ML + 3 SRE

Architecture khác hoàn toàn vì *context khác*. Cùng một bộ nguyên tắc, nhưng áp dụng khác nhau khi Quality Attributes thay đổi.

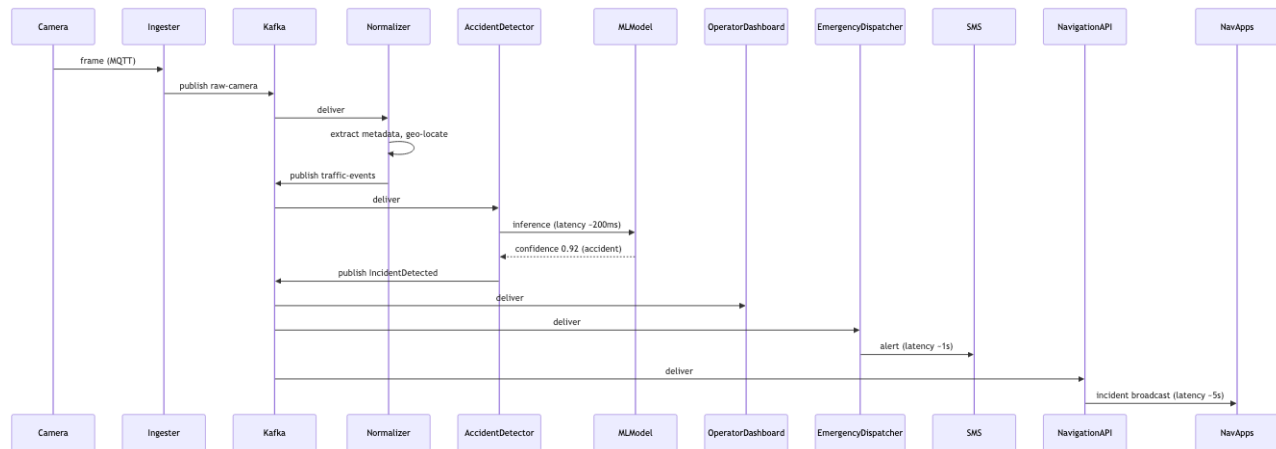
Tóm tắt

Smart City case demonstrates:

- **EDA + Microservices + Microkernel** compose cho high-scale real-time.
- **Kafka** as backbone for streaming.
- **Pipeline pattern** in ingest → normalize → detect.
- **Microkernel** for extensibility (add algorithms).
- **TimescaleDB + BigQuery** for hot/cold data split.

Both UAMS và Smart City đều “correct”, vì serve different contexts. Skill: chọn đúng combination.

Bài tiếp: [Production ML Feature Store](#), case study sát với MLOps và Data Scientist hơn.



Hình 5: Sơ đồ 53

8.4 Production ML Feature Store

Tóm tắt một dòng: Case study này thiết kế một nền tảng production ML cho nhiều team Data Science. Kết luận - Service-based core cho Feature Registry, Model Registry, Serving và Monitoring, kết hợp Pipeline Architecture cho batch features, Event-Driven Architecture cho streaming features và dual store offline/online để giảm training-serving skew.

1. Vì sao case này quan trọng?

Nếu bạn là Data Scientist, đây có lẽ là case study gần công việc thật nhất trong toàn bộ khoá. Rất nhiều team bắt đầu bằng notebook tốt, model tốt, metric tốt. Nhưng khi số lượng model tăng lên, vấn đề không còn là “train model nào” nữa. Vấn đề trở thành:

- Feature bị tính lại ở nhiều nơi, mỗi nơi hơi khác một chút.
- Training dùng logic feature khác serving, gây training-serving skew.
- Model chạy trong production nhưng không ai biết chính xác dùng data version nào.
- Online inference chậm vì mỗi request phải query warehouse.
- Data drift xảy ra nhưng không có dashboard cảnh báo.
- Prediction log thiếu model version, nên không audit được.
- Feature chứa PII bị log ra ngoài vì thiếu data contract.

Nói cách khác, vấn đề đã chuyển từ Data Science thuần sang Software Architecture. Case này giúp bạn thấy cách biến một hệ ML rời rạc thành một nền tảng có boundary, contract, quality attributes và evolution path rõ ràng.

2. Domain và bối cảnh

Một công ty SaaS B2B có 10 team Data Science. Mỗi team xây model cho một use case:

- Churn prediction.
- Lead scoring.
- Fraud detection.
- Recommendation.
- Customer health score.
- Pricing optimization.
- Support ticket priority.

Ban đầu mỗi team tự quản lý pipeline riêng. Sau 18 tháng, công ty gặp vấn đề:

- Có hơn 200 features, nhưng nhiều feature trùng logic.
- Một feature như `active_days_last_30d` được tính 5 kiểu khác nhau.
- Offline training lấy data từ warehouse, online serving tự query database operational.
- Deploy model mới mất 1-2 tuần vì cần backend team viết endpoint riêng.
- Khi business hỏi “vì sao customer này bị score thấp”, không ai truy lại đủ lineage.

Công ty quyết định xây **Production ML Platform** với feature store, model registry, serving API và monitoring.

3. Functional Requirements

Feature management

- Data Scientist đăng ký feature definition.
- Feature có owner, description, schema, source, freshness tier và privacy level.
- Feature được materialize ra offline store cho training.
- Feature quan trọng được materialize ra online store cho serving.
- Feature definition có versioning.

Training support

- Tạo training dataset theo point-in-time correctness.
- Lấy feature version đúng với model version.
- Ghi lại data snapshot, feature list, code version và experiment metadata.
- Đẩy model artifact vào model registry.

Serving support

- Online API nhận entity ID và trả prediction.
- Serving API lookup online features trong latency thấp.
- API log request, feature version, model version, prediction và latency.
- Có canary release và rollback model.

Monitoring and governance

- Theo dõi feature freshness, null rate, drift và schema changes.
- Theo dõi model latency, error rate, prediction distribution và business metric.
- Audit được prediction cụ thể dùng model nào, feature nào, data version nào.
- Enforce policy cho PII và sensitive features.

4. Constraints và scale

Dimension		Estimate
DS/ML teams		10 teams
Models in production		40 models
Registered features	200 features, grow to 1000	
Daily batch records	50M entities	
Event stream	5k events/s average, 20k peak	
Online prediction traffic	2k RPS average, 15k RPS peak	

Dimension	Estimate
Feature lookup target	p95 < 20ms
Prediction target	p95 < 120ms
Batch feature freshness	24h acceptable
Near-realtime freshness	< 5 minutes
Critical realtime freshness	< 10 seconds

Team constraint cũng quan trọng:

- Platform team có 8 engineers.
- Mỗi DS team có 3-6 người.
- Chưa có SRE riêng cho ML platform.
- Công ty dùng cloud warehouse, object storage, Kubernetes nhẹ, CI/CD cơ bản.

5. Identify Quality Attributes

Top QA

Priority	QA	Target
Must	Training-serving consistency	100% online features có cùng definition với offline training
Must	Online latency	p95 feature lookup < 20ms, p95 prediction < 120ms
Must	Freshness tiering	Batch 24h, near-realtime 5m, realtime 10s
Must	Auditability	100% predictions trace được model version + feature versions
Must	Privacy	0 PII leak trong logs, policy enforced ở feature level
Should	Reusability	70% features dùng lại bởi ít nhất 2 models sau 12 tháng
Should	Deployability	Rollback model trong < 10 phút
Should	Cost control	Serving cost < \$0.20 / 1000 predictions
Should	Observability	Dashboard cho freshness, drift, latency, null rate

Operationalize

QA	Metric	Tooling
Consistency	Feature definition hash match giữa training và serving	Feature Registry
Latency	p50, p95, p99 per endpoint	Tracing + API metrics
Freshness	Age của feature value theo entity	Online store metadata
Auditability	Prediction log completeness	Audit query + data lake
Privacy	PII scan logs, policy check	DLP scanner + access control

QA	Metric	Tooling
Reusability	Feature reuse count	Registry analytics
Rollback	Time from rollback trigger to stable old version	Deployment metrics
Drift	PSI, KS test, null rate, range violations	Monitoring jobs

6. Architecture style decision

Options considered

Option	Lợi ích	Chi phí
Mỗi team tự build pipeline	Tự do cao, không cần platform ban đầu	Duplicate logic, skew, audit kém, cost tăng
Modular monolith ML platform	Đơn giản, dễ build MVP	Khó scale team ownership khi platform lớn
Service-based platform	Boundary rõ, ops cost vừa phải	Cần API contracts và service ownership
Full microservices	Independence cao	Ops cost quá cao cho team 8 người

Decision

Chọn **Service-based Architecture** cho core platform, overlay **Pipeline Architecture** và **Event-Driven Architecture** cho data flow.

Lý do:

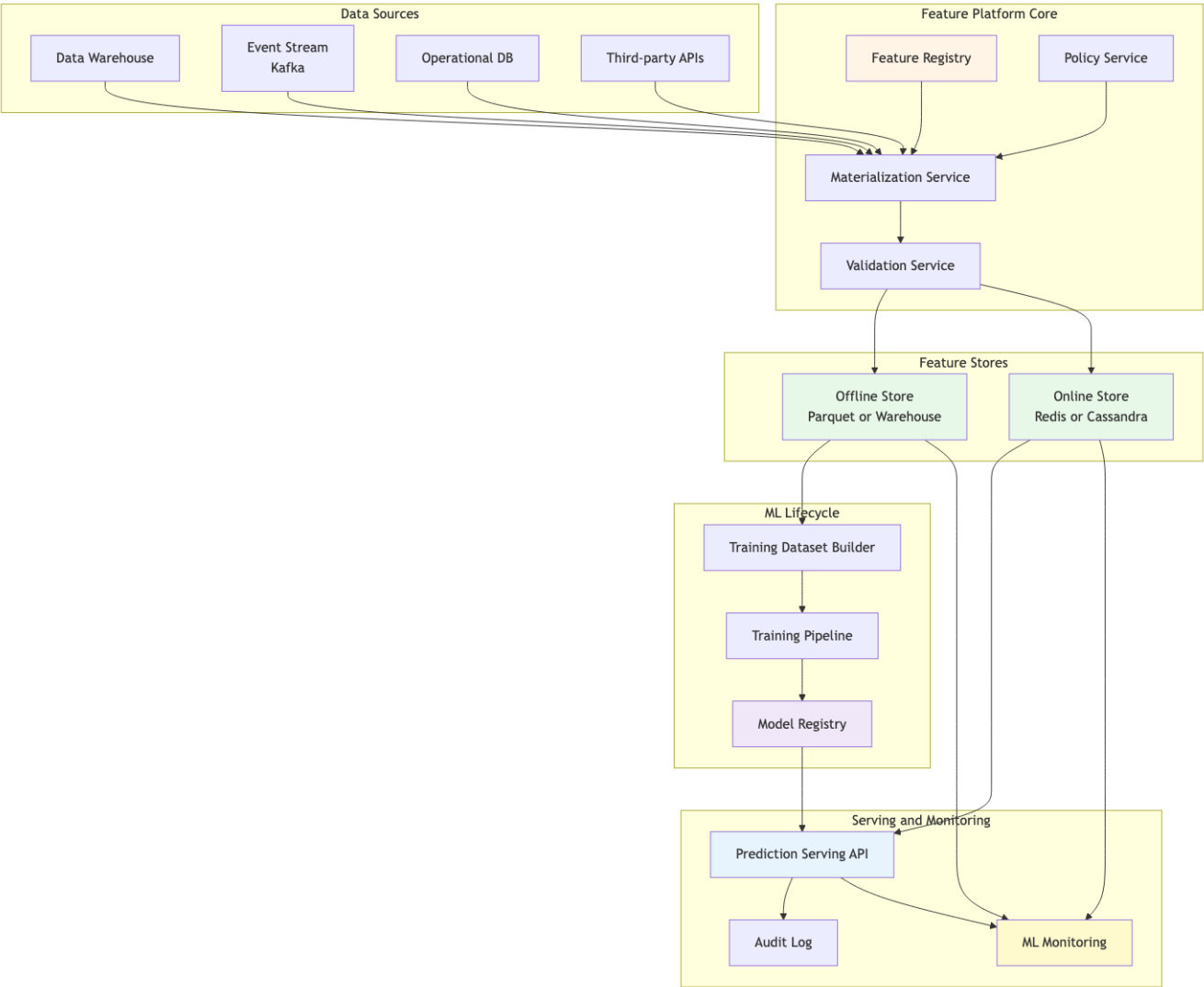
- Team platform 8 người không đủ để vận hành full microservices.
- Domain có 5-7 bounded contexts rõ: Feature Registry, Materialization, Offline Store, Online Store, Model Registry, Serving, Monitoring.
- Data flow bản chất là pipeline, đặc biệt cho batch feature và training dataset.
- Streaming features cần event-driven, nhưng không phải mọi feature đều cần streaming.
- Service-based cho phép deploy độc lập ở mức coarse-grained mà không tạo 30 service nhỏ khó vận hành.

7. High-level Architecture

Điểm quan trọng: đây không phải một style duy nhất. Hệ thực tế thường là **style mix**:

- Service-based cho boundary giữa platform services.
- Pipeline cho materialization và training.
- Event-driven cho streaming feature updates và prediction logs.
- Layered bên trong từng service.
- Microkernel nhẹ cho plugin feature transformations hoặc custom metrics.

8. Component decomposition

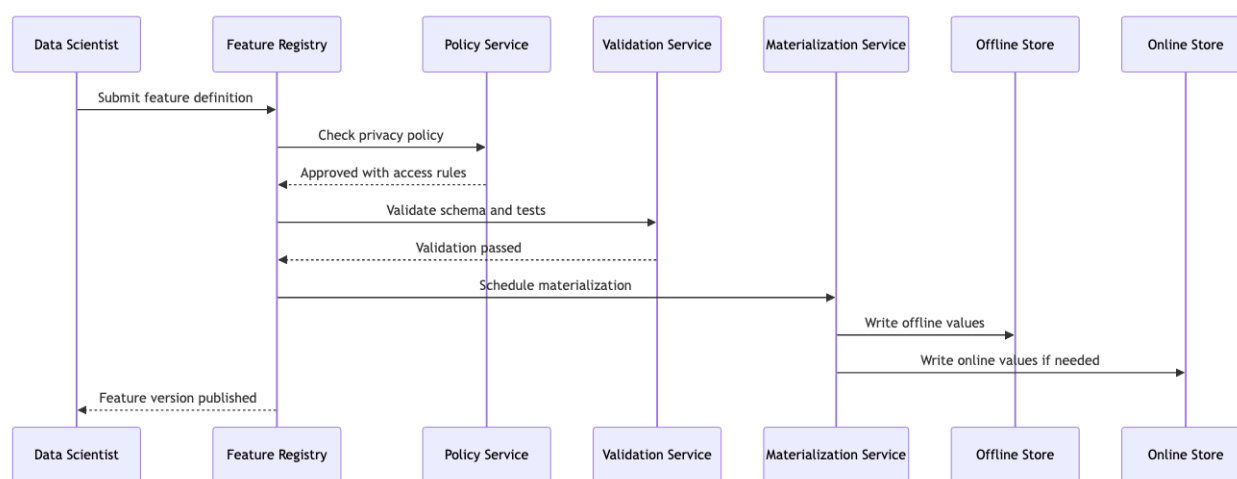


Hình 6: Sơ đồ 54

Component	Responsibility	Owner	State
Feature Registry	Feature metadata, schema, owner, version	Platform team	Metadata DB
Policy Service	Privacy level, access control, PII rules	Security + Platform	Policy DB
Materialization Service	Run batch/stream jobs to compute features	Data Platform	Job metadata
Validation Service	Schema, null rate, range, freshness checks	Platform team	Validation results
Offline Store	Historical feature values for training	Data Platform	Warehouse/Object storage
Online Store	Low-latency feature values for serving	Platform team	Redis/Cassandra
Training Dataset Builder	Point-in-time join for training	ML Platform	Dataset metadata
Model Registry	Model artifact, version, metrics, lifecycle	ML Platform	Registry DB/Object storage
Serving API	Prediction endpoint, feature lookup, model runtime	ML Platform	Runtime cache
Monitoring	Drift, freshness, latency, business KPIs	ML Platform	Metrics DB
Audit Log	Immutable prediction trace	Platform + Compliance	Data lake

9. Key runtime flows

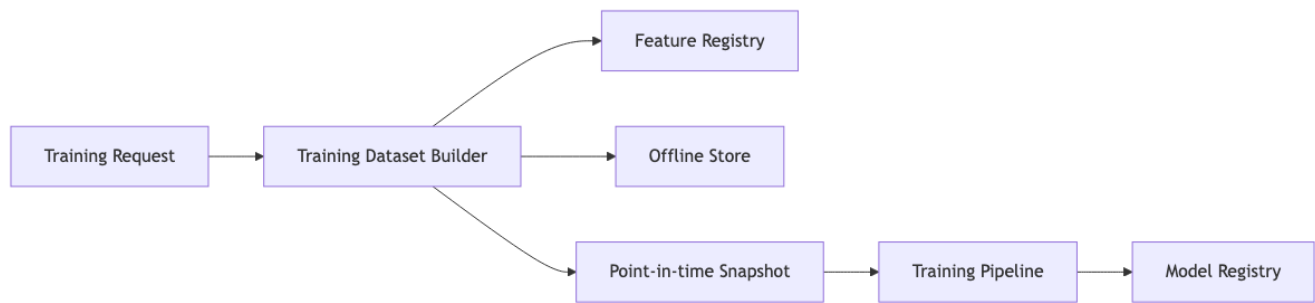
Flow A: Register a feature



Hình 7: Sơ đồ 55

Architecture insight: feature registration is not just a form. It is a governance workflow. Privacy, schema, freshness and ownership must be enforced before feature becomes reusable.

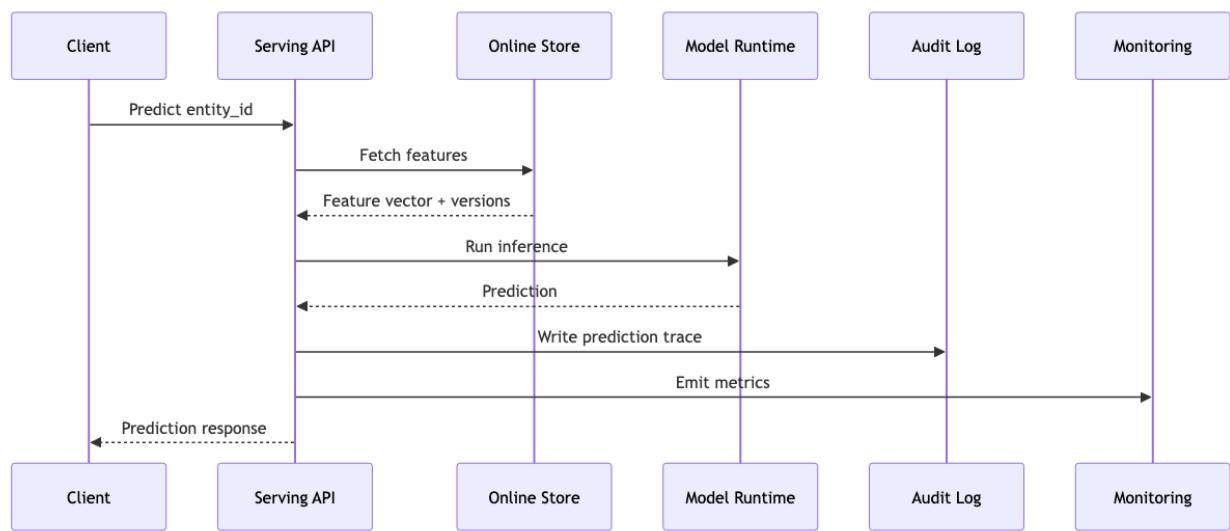
Flow B: Build training dataset



Hình 8: Sơ đồ 56

Point-in-time correctness là điểm rất dễ bị bỏ qua. Nếu bạn train churn model cho ngày 1/5, bạn không được dùng feature được tính bằng dữ liệu sau ngày 1/5. Nếu không, model có data leakage và validation metric sẽ ảo.

Flow C: Online prediction



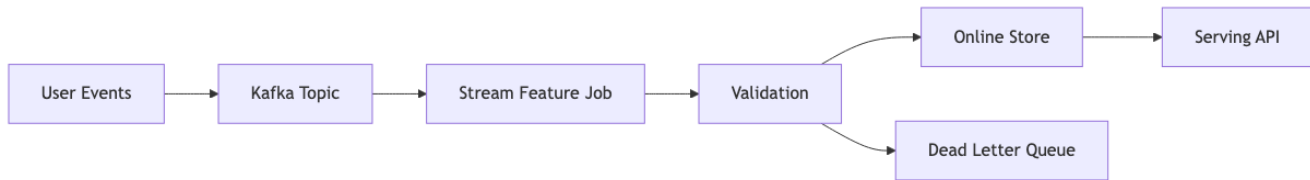
Hình 9: Sơ đồ 57

Latency budget ví dụ:

Step	Target
API auth + validation	10ms
Feature lookup	20ms
Model inference	50ms
Logging async enqueue	5ms
Network + serialization	20ms
Buffer	15ms
Total p95	120ms

Nếu feature lookup query warehouse trực tiếp, target 20ms gần như bất khả thi. Đây là lý do online store tồn tại.

Flow D: Streaming feature update



Hình 10: Sơ đồ 58

Streaming chỉ dùng cho feature thật sự cần freshness thấp. Những feature dùng cho churn campaign daily không cần đi qua flow này. Architecture tốt không biến mọi thứ thành realtime.

10. Data contracts

Feature definition cần đủ rõ để cả training và serving dùng chung.

Ví dụ metadata:

```

name: active_days_last_30d
version: 3
owner: growth_ml_team
entity: account_id
description: Number of active product usage days in the last 30 days
source: product_events
freshness_tier: daily
privacy_level: internal
value_type: integer
nullable: false
range:
  min: 0
  max: 30
offline_store: warehouse.features_account_daily
online_store: redis.account_features
created_by: growth_ml_team
  
```

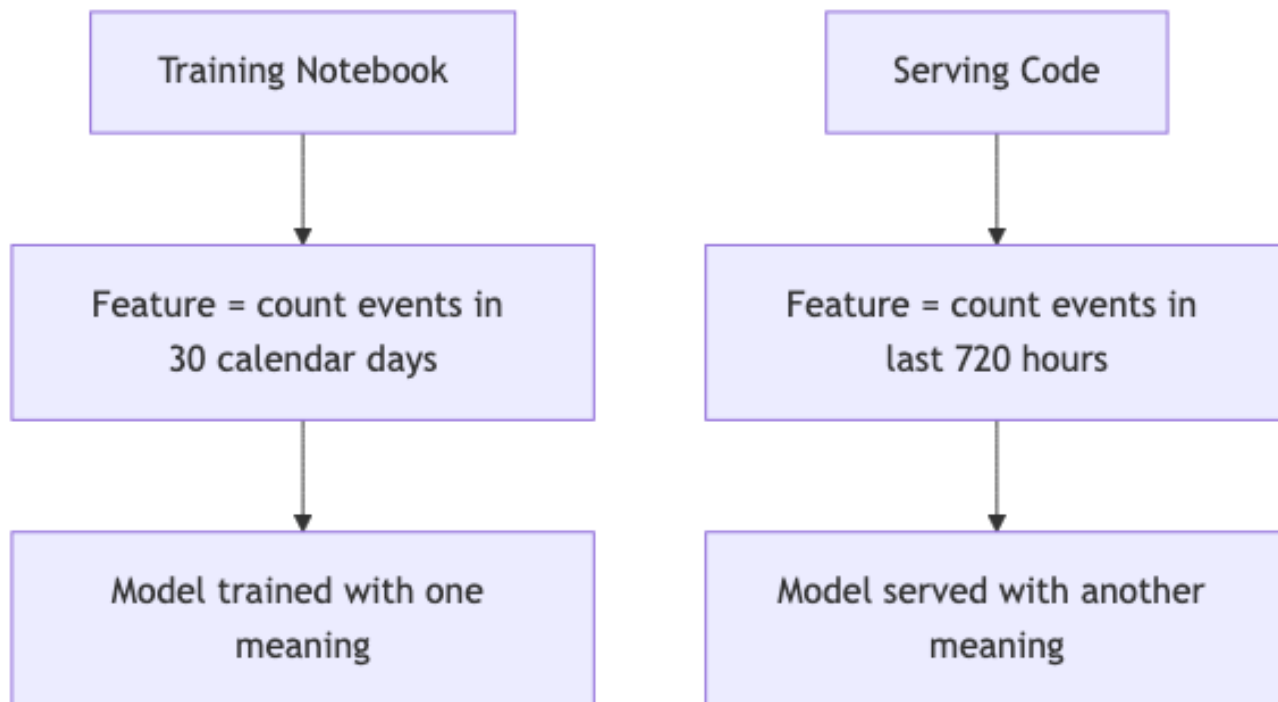
Điểm quan trọng không phải YAML đẹp. Điểm quan trọng là contract này trở thành source of truth. Training không tự viết lại logic. Serving không tự suy diễn schema. Monitoring biết range đúng để detect anomaly.

11. Training-serving skew

Training-serving skew là lỗi trung tâm mà feature store muốn giảm.

Skew scenario

Hai định nghĩa nghe gần giống nhau, nhưng kết quả có thể khác. Model validation tốt nhưng production tệ. Debug rất khó vì không có exception nào xảy ra.



Hình 11: Sơ đồ 59

Architecture fix

- Feature definition sống trong Feature Registry.
- Offline và online materialization dùng cùng transformation source.
- Model registry lưu feature list + feature versions.
- Serving API check feature version compatibility trước khi load model.
- Monitoring so sánh offline distribution và online distribution.

12. Privacy and governance

Feature store rất dễ trở thành nơi tập trung dữ liệu nhạy cảm. Vì vậy privacy không thể là phần thêm sau.

Policy examples

Feature type	Policy
PII raw email, phone	Không serve trực tiếp cho model nếu không có approval
Financial data	Encrypt at rest, restrict access, audit read
Behavioral events	Aggregate before online serving nếu có thể
Sensitive demographic	Cần fairness review trước khi dùng
Free text notes	Không log raw text trong prediction trace

Design implication

Policy Service phải nằm trong path feature registration và feature access. Nếu chỉ dựa vào convention, sớm muộn sẽ có team log nhầm hoặc train nhầm feature nhạy cảm.

13. Observability design

Production ML monitoring không chỉ là CPU và memory.

Layer	Metrics
Data ingestion	event lag, dropped events, schema changes
Feature materialization	job duration, freshness, null rate, range violation
Online store	lookup latency, hit rate, stale values, error rate
Serving API	p95 latency, RPS, model error, timeout, rollback count
Model quality	prediction distribution, drift, calibration, delayed labels
Business	churn intervention success, fraud false positives, revenue lift

Một dashboard tốt phải giúp trả lời: lỗi nằm ở data, feature, model, serving hay business feedback loop?

14. Failure modes and mitigations

Failure	Symptom	Mitigation
Schema drift	Feature job fail hoặc null tăng	Schema validation + alert
Stale online feature	Prediction dùng data cũ	Freshness metadata + fallback
Duplicate event	Feature count bị inflate	Idempotency key + dedup window
Online store down	Prediction timeout	Graceful degradation, cached defaults, circuit breaker
Bad model release	Business metric giảm	Canary + shadow deployment + rollback
PII leak in logs	Compliance incident	Log redaction + DLP scan
Training-serving skew	Offline tốt, online tệ	Shared feature definition + compatibility check
Label delay	Monitoring quality chậm	Proxy metrics + delayed evaluation job

15. ADR examples

ADR 1: Use dual offline and online feature stores

Context: Training cần historical features và point-in-time join. Online serving cần lookup thấp hơn 20ms.

Decision: Dùng offline store cho training và online store cho serving. Feature Registry quản lý cùng definition và versions cho cả hai.

Consequences:

- Giảm training-serving skew.
- Tăng complexity vì cần materialization hai đích.
- Cần consistency check giữa offline và online values.

ADR 2: Support freshness tiers instead of realtime for all features

Context: Một số use case cần realtime, nhưng churn và health score chỉ cần daily.

Decision: Mỗi feature khai báo freshness tier: daily, hourly, near-realtime, realtime.

Consequences:

- Giảm cost đáng kể.
- Platform phức tạp hơn vì có nhiều execution mode.
- DS phải hiểu freshness requirement khi đăng ký feature.

ADR 3: Choose service-based core, not full microservices

Context: Platform team có 8 engineers, domain chưa đủ lớn để justify 30 services.

Decision: Dùng 6-8 coarse-grained services, shared metadata DB có schema boundary rõ.

Consequences:

- Ops cost vừa phải.
- Boundary đủ rõ cho team ownership.
- Khi platform lớn hơn, có thể tách dần bằng Strangler Fig.

16. Cost model rough estimate

Component	Monthly cost estimate
Metadata DB	\$300-800
Offline store storage + query	\$3000-8000
Online store	\$2000-6000
Batch compute	\$3000-10000
Streaming compute	\$2000-8000
Serving API compute	\$2000-7000
Monitoring/log storage	\$1000-4000
Total	\$13k-44k/month

Cost range lớn vì phụ thuộc số feature, freshness tier và traffic. Bài học kiến trúc: freshness và latency là hai driver cost lớn nhất. Đừng realtime hoá feature nếu business không cần.

17. Evolution roadmap

Phase 1: Batch-first feature platform

- Feature Registry.
- Offline Store.
- Batch materialization.
- Training Dataset Builder.
- Model Registry cơ bản.

Dùng khi use case chủ yếu là churn, lead scoring, campaign, reporting. Đây là phase rẻ và ít rủi ro.

Phase 2: Online serving

- Online Store.
- Serving API.
- Prediction log.
- Rollback model.
- Basic monitoring.

Dùng khi cần prediction trong app hoặc API, nhưng chưa cần streaming features phức tạp.

Phase 3: Streaming and governance

- Kafka/Flink streaming feature jobs.
- Freshness monitoring.
- Drift dashboard.
- Policy Service cho PII.
- Canary/shadow deployment.

Dùng khi fraud detection, realtime recommendation hoặc personalization thật sự cần freshness thấp.

18. What a Data Scientist should learn from this case

Nếu bạn đến từ Data Science, đừng chỉ nhìn feature store như một tool. Hãy nhìn nó như một architecture boundary.

Feature store trả lời các câu hỏi kiến trúc:

- Feature definition sống ở đâu?
- Ai own feature đó?
- Training và serving có dùng cùng logic không?
- Feature có freshness SLA nào?
- Feature có chứa dữ liệu nhạy cảm không?
- Model version nào dùng feature version nào?
- Khi prediction sai, truy lại input thế nào?

Khi bạn trả lời được các câu hỏi này, bạn không chỉ là người train model. Bạn đang thiết kế một phần của production system.

19. Self-check

1. Với model bạn từng làm, 5 features quan trọng nhất có owner và version rõ không?
2. Training và serving có dùng cùng feature transformation không?
3. Nếu một prediction bị khiếu nại, bạn truy lại được model version và feature values không?
4. Feature nào thật sự cần realtime, feature nào chỉ cần daily?
5. Nếu online store down, hệ thống fail đóng, fail mở hay dùng fallback?
6. Có feature nào chứa PII nhưng đang bị log ra prediction trace không?
7. Nếu thêm model mới, bạn cần sửa Serving API hay chỉ đăng ký model artifact mới?

20. Tóm tắt

- Production ML Platform là bài toán Software Architecture, không chỉ là bài toán modelling.

- Feature Store giảm training-serving skew bằng cách centralize feature definition, version và materialization.
- Offline store phục vụ training, online store phục vụ low-latency serving.
- Không phải feature nào cũng cần realtime. Freshness tiers giúp cân bằng cost và value.
- Service-based core phù hợp hơn full microservices khi platform team còn nhỏ.
- Pipeline Architecture phù hợp batch feature và training dataset.
- Event-Driven Architecture phù hợp streaming feature, prediction logs và monitoring.
- Auditability, privacy, observability và rollback phải được thiết kế từ đầu.

Bài tiếp: [Bài tập tổng hợp](#), nơi bạn tự thiết kế architecture cho nhiều bối cảnh khác nhau.

8.5 Bài tập tổng hợp

Tóm tắt một dòng: 8 bài tập tự practice. Mỗi bài có context, requirements, và hints để bạn lập luận theo framework từ Phần 1-7. Không có “đáp án đúng” - chỉ có “đáp án hợp lý cho context”.

Cách làm

Cho mỗi bài tập:

1. **Đọc context** + functional requirements.
2. **Identify Top 5-7 Quality Attributes** (Phần 4).
3. **Quyết định monolithic vs distributed** (Bài 5.2).
4. **Choose architecture style** (Phần 5-6).
5. **Decompose components** (Bài 4.4, 3.4).
6. **Draft 3-5 ADRs** cho key decisions (Bài 7.2).
7. **Vẽ C4 Context + Container diagrams** (Bài 7.2).
8. **Identify trade-offs hy sinh** (Bài 3.3).

Time budget: 2-3 giờ mỗi bài.

Tốt nhất là làm với 1-2 đồng nghiệp + present cho nhau. SA là kỹ năng *socialized*, debate giúp identify weak reasoning.

Bài 1: Online Food Delivery Platform

Context

Startup Việt Nam build platform giao đồ ăn (như GrabFood, ShopeeFood).

- Phase 1 (6 tháng): launch tại HCM, ~1000 restaurants, ~50k customer, ~500 driver.
- Phase 2 (12 tháng): expand 5 thành phố lớn, 10x scale.
- Phase 3 (24 tháng): national, 100x scale phase 1.

Functional requirements

- Customer browse restaurant + menu, place order.
- Restaurant receive order, confirm/reject.
- Driver được assigned, pickup, deliver.
- Real-time tracking driver location for customer.
- Payment (digital wallet, COD).
- Rating + review.

Constraints

- Initial team: 8 engineers.
- Budget: < \$5k/month infrastructure phase 1.

Hints

- **Scale trajectory:** design cho phase 1, planned migration to phase 2.
- **Real-time tracking:** WebSocket + location streaming.

- **Order workflow:** state machine (placed ☐ confirmed ☐ preparing ☐ ready ☐ picked ☐ delivered).
- **Multi-sided platform:** 3 user types (customer, restaurant, driver), consider BFF pattern.

Things to consider

- Monolithic phase 1, migrate phase 2? Hay service-based từ đầu?
- DB: PostgreSQL? Add Redis? Add Kafka?
- Payment: build hay integrate (Stripe/local gateway)?
- Notification: push notification setup?

Output expected

- C4 Context + Container.
- Top 5 QA + targets.
- 4-5 ADRs.
- Trade-off reflection.

Bài 2: B2B SaaS for Property Management

Context

SaaS giúp building owner quản lý apartment complex (như Yardi, RealPage but VN-focused).

- Khách hàng: building owner / management company (100-2000 apartments per customer).
- Target: 50 customers in year 1, 500 in year 3.

Functional requirements

- Tenant management (lease, contract, profile).
- Rent collection (recurring billing, payment tracking).
- Maintenance ticket (tenant submit ☐ assigned to staff ☐ resolve).
- Financial reporting (collected, outstanding, occupancy rate).
- Customer admin portal + Tenant mobile app.

Constraints

- Multi-tenancy (khách hàng A không thấy data khách hàng B).
- Customization: each building can have custom rules (vd: late fee policy).
- Compliance: GDPR-like privacy regulation.

Hints

- **Multi-tenancy:** shared DB + tenant_id column? Database-per-tenant? Schema-per-tenant?
- **Customization:** rule engine? Plug-in?
- **Reporting:** separate read replica? CQRS?
- **Mobile app:** BFF for tenant?

Things to consider

- Scale 50-500 customers, service-based likely sufficient.
- Multi-tenancy strategy có ảnh hưởng lớn đến architecture.
- Compliance audit log.

Output expected

- Same as Bài 1.
 - Bonus: design multi-tenancy schema in detail.
-

Bài 3: Real-time Stock Trading Platform**Context**

Startup fintech build platform giao dịch chứng khoán (Robinhood-style).

- Target users: 100k retail trader Vietnam.
- Peak hours: 9:00-15:00 weekdays.
- Connect to local exchange (HOSE, HNX) via FIX protocol.

Functional requirements

- View real-time price ticker.
- Place buy/sell order.
- Portfolio tracking.
- Historical price chart (1-min, 5-min, daily candles).
- News feed.
- Account & money transfer.

Constraints

- Compliance: SBV regulation, audit log mọi transaction.
- Latency critical: order execution < 100ms.
- Data integrity: 100% accurate (financial).
- Uptime peak hours: 99.99%.

Hints

- **Price ticker:** WebSocket streaming. Pub-sub pattern. Redis pub/sub or Kafka?
- **Order matching engine:** critical path, likely in-house cho fairness/latency.
- **Historical data:** time-series DB (TimescaleDB, InfluxDB).
- **Compliance:** event sourcing (cho audit + replay)?

Things to consider

- This is “harder” than typical app, financial, real-time, regulated.
- Space-Based architecture có relevant không (for matching engine)?
- Disaster recovery critical.

Output expected

- Same as before + DR/failover plan.
 - Critical: explain latency budget breakdown.
-

Bài 4: IoT Manufacturing Monitoring**Context**

Manufacturing company want monitor 50 factories worldwide. Each factory has 200 machines với sensor (temperature, vibration, power, ...).

Functional requirements

- Ingest sensor data (10k sensors total, each pings every 5s = 2000 events/s).
- Real-time dashboard cho factory manager.
- Anomaly detection (predictive maintenance).
- Historical analytics (5 year retention).
- Alert system (SMS + email + integration với existing ticketing).

Constraints

- Some factories có poor internet, edge processing cần thiết.
- Latency: anomaly detection within 60s.
- Cost-sensitive (industrial budget, not tech startup).
- Compliance: ISO 27001.

Hints

- **Edge processing**: do detection locally, sync results to central.
- **Cloud + on-prem hybrid**.
- **Time-series storage**: InfluxDB, TimescaleDB.
- **Anomaly detection**: rule + ML.

Things to consider

- Hybrid (edge + cloud) is rare but appropriate here.
 - Bandwidth limited $\square$ compress + batch.
 - Lambda architecture (batch + stream) possible.
-

Bài 5: Multi-region E-commerce Platform**Context**

Major e-commerce expand từ Vietnam ra Southeast Asia (Thailand, Indonesia, Philippines, Malaysia).

- Current: monolith Vietnam, 10M users.
- Target: multi-region, 50M users across SEA.
- Constraints: comply local data residency law (each country's data must stay in-country).

Functional requirements

- Catalog (products country-specific + global).
- Cart, checkout, payment (country-specific payment methods).
- Inventory (country-specific warehouses).
- Search (country-specific language).
- Recommendations (cross-region OK).

Constraints

- Latency: page load < 2s in each country.
- Data residency: hard requirement.
- Existing monolith: 5-year-old codebase, can't rewrite.

Hints

- **Migration strategy:** Strangler Fig (Bài 5.2).
- **Multi-region deployment:** each country has DB; some services shared globally (recommendations).
- **CDN:** critical for asset performance.
- **Data residency:** dictate database location, search index, cache.

Things to consider

- This is the hardest exercise, combine migration + multi-region + compliance.
- Hybrid: legacy monolith + new microservices.
- Plan over 24-36 months.

Bài 6: Churn Prediction Platform**Context**

Một công ty SaaS muốn dự đoán khách hàng có nguy cơ churn để đội Customer Success can thiệp sớm. Data đến từ product usage events, billing history, support tickets và CRM notes.

Functional requirements

- Tính churn score cho mỗi account.
- Export score sang CRM.
- Lưu prediction history để phân tích hiệu quả campaign.
- Cho phép data scientist deploy model version mới.

Constraints

- Business chỉ gọi khách theo daily campaign mỗi sáng.
- Data chứa PII và thông tin billing.
- Team ML có 5 người, chưa có SRE riêng.

Questions

1. Chọn batch inference hay online inference? Vì sao?
2. Quality Attributes top 5 là gì?
3. Có cần Kafka không, hay Airflow batch đủ?
4. Model registry cần lưu metadata nào?
5. Vẽ C&C View cho daily scoring flow.

Bài 7: Real-time Fraud Detection

Context

Một ví điện tử cần detect giao dịch gian lận trong lúc user thanh toán. Hệ thống phải trả quyết định approve, reject hoặc manual review rất nhanh.

Functional requirements

- Nhận transaction event từ payment gateway.
- Tính features gần realtime.
- Gọi model để tạo fraud score.
- Gửi quyết định về payment flow.
- Lưu audit log cho mỗi quyết định.

Constraints

- p95 latency end-to-end < 200ms.
- Cần explainability ở mức reason code.
- Transaction duplicate có thể xảy ra.
- Label fraud về trễ vài ngày.

Questions

1. Feature nào precompute, feature nào tính request-time?
2. REST sync, Kafka async hay hybrid?
3. Dead-letter queue xử lý event lỗi ra sao?
4. Monitoring cần đo drift, latency và false positive như thế nào?
5. ADR quan trọng nhất trong hệ này là gì?

Bài 8: Feature Store cho ML Platform

Context

Nhiều team DS trong công ty tính feature riêng, dẫn tới duplicate logic và training-serving skew. Công ty muốn xây feature store dùng chung.

Functional requirements

- Đăng ký feature definition.
- Materialize feature cho offline training.
- Serve online feature cho low-latency inference.
- Version feature và tracking lineage.

Constraints

- Một số feature cần freshness < 5 phút.
- Một số feature chỉ cần daily batch.
- Data source gồm warehouse, event stream và third-party API.
- Phải kiểm soát quyền truy cập PII.

Questions

1. Component chính của feature store là gì?
2. Boundary giữa feature store và training pipeline nằm ở đâu?
3. QA nào quyết định online store cần Redis/Cassandra hay không?
4. Data lineage được document ở view nào?
5. Khi feature definition đổi, model cũ xử lý ra sao?

Reflection questions

Sau khi làm 5 bài:

1. Bài nào dễ nhất? Vì sao?
2. Bài nào khó nhất? Vì sao?
3. Style nào bạn dùng nhiều nhất across exercises? Có pattern không?
4. QA nào hay được prioritize? (security, scalability, ...)
5. Quyết định bạn ít confident nhất? Cần học gì thêm?

Câu trả lời tiết lộ strengths + gaps của bạn. Use it to plan further learning.

Discussion với mentor (nếu có)

Show bài tập của bạn cho:

- Senior architect tại công ty.
- Mentor từ tech community.
- Bạn cùng nhóm học.

Hỏi:

- “Đâu là điểm tôi missing?”
- “Bạn would do differently cái nào?”
- “Quyết định nào bạn would push back?”

Feedback từ người có kinh nghiệm thực tế > self-study 10 giờ.

Tóm tắt

- 8 bài tập từ easy (food delivery) đến hard (multi-region migration, real-time fraud, feature store).
- Apply framework end-to-end.
- Discuss với mentor để identify gap.

Tổng kết Phần 8 và toàn khoá

Hết Phần 8. Hết khoá.

Tóm tắt journey:

- **Phần 1:** framework tư duy.
- **Phần 2:** foundation principles (SOLID, cohesion-coupling).

- **Phần 3:** tư duy architect (vs designer).
- **Phần 4:** cái cần tối ưu (QA).
- **Phần 5-6:** 9 styles để chọn.
- **Phần 7:** cách document.
- **Phần 8:** áp end-to-end vào case thực.

Bạn đã có toolkit đầy đủ. Bước tiếp theo là *practice* trên dự án thật của bạn. Architecture là kỹ năng *applied*: chỉ đọc không đủ.

Suggested next steps:

1. Đọc lại [Course Summary](#) để consolidate.
2. Apply framework vào 1 dự án thật bạn đang làm.
3. Đọc thêm: *Fundamentals of Software Architecture* (Mark Richards), *Software Architecture in Practice* (Bass-Clements-Kazman).
4. Practice qua [exam checklist](#) nếu chuẩn bị phỏng vấn.

Chúc bạn trở thành software architect xuất sắc.